

Schema Reference

AI-Based Examination Platform

Persistence	PostgreSQL · SQLAlchemy 2.0 declarative · Alembic
Wire contract	Pydantic v2 · FastAPI
Source	<code>backend/app/db/models/</code> · <code>backend/app/schemas/</code>
Scope	13 tables · 10 native enum types · 60 API models · 3 migrations

- | | |
|---|--|
| 1 Conventions & shared mixins | 8 Enum types |
| 2 Identity & access | 9 Referential integrity map |
| 3 Question bank | 10 API schemas — auth & admin |
| 4 Exams | 11 API schemas — questions & exams |
| 5 Sessions & answers | 12 API schemas — sessions, grading, proctoring |
| 6 Proctoring | 13 Migrations |
| 7 Grading & results | |

1 Conventions & shared mixins

Every table inherits a declarative Base carrying an explicit constraint naming convention, so Alembic autogenerate produces stable, revertible names. `backend/app/db/base.py`

Naming convention

KEY	TEMPLATE	YIELDS
pk	<code>pk_%(table_name)s</code>	<code>pk_users</code>
fk	<code>fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s</code>	<code>fk_answers_session_id_exam_sessions</code>
uq	<code>uq_%(table_name)s_%(column_0_name)s</code>	<code>uq_subjects_code</code>
ix	<code>ix_%(column_0_label)s</code>	<code>ix_users_email</code>
ck	<code>ck_%(table_name)s_%(constraint_name)s</code>	—

Mixins

MIXIN	COLUMN	TYPE	DEFINITION
UUIDPrimaryKeyMixin	<code>id</code>	UUID	Primary key, <code>default=uuid.uuid4</code> — generated client-side
TimestampMixin	<code>created_at</code>	timestampz	NOT NULL, <code>server_default=now()</code>
	<code>updated_at</code>	timestampz	NOT NULL, <code>server_default=now()</code> , <code>onupdate=now()</code>

In the tables that follow, `timestamps` marks a table carrying both `created_at` and `updated_at`; `no timestamps` marks one that omits the mixin deliberately — join tables and the high-volume proctoring log. Every table carries `id`.

2 Identity & access

Four independent gates. `is_active` is account state; `access_status` is the admin gate over examiners; `login_access_requests` is the gate over a candidate's ability to sign in at all; `exam_enrollments` is the per-exam layer on top.

users TIMESTAMPS

backend/app/db/models/user.py

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK, <code>uuid4</code>
email	<code>varchar(255)</code>	NO	UNIQUE, INDEX — the login identity, never user-editable
password_hash	<code>varchar(255)</code>	NO	Never leaves the server
first_name	<code>varchar(80)</code>	NO	<code>""</code> — source of truth for greetings and sorting
last_name	<code>varchar(80)</code>	NO	<code>""</code>
full_name	<code>varchar(150)</code>	NO	Derived, kept in sync by <code>set_name()</code> so existing queries, exports and UI that read it keep working
role	<code>user_role</code>	NO	INDEX
is_active	<code>bool</code>	NO	<code>true</code>
access_status	<code>access_status</code>	NO	<code>pending</code> , INDEX — an examiner is inert until an admin approves; candidates are approved at registration
access_changed_at	<code>timestampz</code>	yes	
access_changed_by_id	UUID	yes	FK <code>users.id</code> SET NULL — self-referential
access_note	<code>text</code>	yes	Reviewer's rationale
last_login_at	<code>timestampz</code>	yes	

Derived `is_approved` → `access_status` is APPROVED · `display_first_name` → `first_name`, falling back to the first token of `full_name`

Relationships `exam_sessions`, `login_access_request` (1:1), `enrollments` — all delete-orphan · `exams_created` · `access_changed_by` (self)

login_access_requests TIMESTAMPS

backend/app/db/models/login_access.py

Permission to *use* the platform, deliberately separate from account state. A candidate's account is created active and stays active; this row is what an administrator — or an examiner authorised over them — opens. Approval is durable state, not an event log: later logins reuse the same row and never queue again unless someone revokes it, which moves the row back to rejected with a note.

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
candidate_id	UUID	NO	FK <code>users.id</code> CASCADE · UNIQUE · INDEX — one standing request per candidate
status	<code>login_access_status</code>	NO	<code>pending</code> , INDEX
requested_at	<code>timestampz</code>	NO	Raised on the first sign-in attempt with valid credentials
reviewed_at	<code>timestampz</code>	yes	
reviewed_by_id	UUID	yes	FK <code>users.id</code> SET NULL

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
review_note	text	yes	Capped at 500 chars by the API layer

exam_enrollments

NO TIMESTAMPS

backend/app/db/models/enrollment.py

Approved login is permission to use the platform; it is not permission to sit every paper on it. A candidate sees and can start only the exams they are enrolled in. This table also defines an examiner's authority over a candidate: an examiner may review a candidate's login request only if that candidate is enrolled in one of the examiner's own exams.

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
exam_id	UUID	NO	FK exams.id CASCADE · INDEX
candidate_id	UUID	NO	FK users.id CASCADE · INDEX
assigned_by_id	UUID	yes	FK users.id SET NULL
assigned_at	timestampz	NO	

UNIQUE

uq_exam_enrollment (exam_id, candidate_id)

3 Question bank

Subjects own questions; questions own their options. Both parent links are RESTRICT — a subject or question that any exam or answer references cannot be deleted, only deactivated.

subjects TIMESTAMPS

backend/app/db/models/question.py

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
code	varchar(32)	NO	UNIQUE, INDEX
name	varchar(150)	NO	
description	text	yes	

questions TIMESTAMPS

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
subject_id	UUID	NO	FK subjects.id RESTRICT
question_type	question_type	NO	
difficulty	difficulty	NO	medium
body	text	NO	Min 3 chars at the API layer
model_answer	text	yes	Reference answer for subjective types; also shown to the examiner while grading
rubric	jsonb	yes	Free-form, handed to the grader — {"criteria": [{"name": "...", "weight": ...}]}
min_words	int	yes	Written answers only. Enforced server-side on every autosave and shown as a live counter in the runner, so a candidate is never surprised at submit time
max_words	int	yes	
marks	float	NO	1.0, must be > 0
negative_marks	float	NO	0.0, must be ≥ 0
tags	jsonb	yes	list[str]
is_active	bool	NO	true, INDEX — soft delete for the bank
created_by_id	UUID	yes	FK users.id SET NULL

INDEX ix_questions_pool_lookup (subject_id, question_type, difficulty) — the composite that serves exam pool selection

question_options NO TIMESTAMPS

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
question_id	UUID	NO	FK questions.id CASCADE
text	text	NO	

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
is_correct	bool	NO	false — the answer key . Never serialised except through <code>OptionOutWithAnswer</code> / <code>QuestionOutFull</code>
order_index	int	NO	0 — the relationship loads ordered by this

4 Exams

An exam holds its window, its selection rules, and two JSONB config blobs. `exam_questions` is the pool it draws from; a candidate's actual paper is a deterministic subset.

exams TIMESTAMPS

backend/app/db/models/exam.py

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
subject_id	UUID	NO	FK subjects.id RESTRICT
title	varchar(200)	NO	
description	text	yes	
duration_minutes	int	NO	1–600; must fit inside the window
starts_at	timestamptz	NO	
ends_at	timestamptz	NO	Must be after starts_at
selection_rules	jsonb	NO	{ } — { "rules": [{ "question_type": "mcq", "difficulty": "easy", "count": 10 }] }
randomize	bool	NO	true
shuffle_options	bool	NO	true
negative_marking	bool	NO	false
paper_salt	varchar(64)	NO	Stable per-exam salt, so the deterministic paper seed cannot be guessed from ids alone
proctor_config	jsonb	NO	DEFAULT_PROCTOR_CONFIG — see below
grading_config	jsonb	NO	DEFAULT_GRADING_CONFIG — see below
status	exam_status	NO	draft, INDEX
results_published	bool	NO	false
created_by_id	UUID	yes	FK users.id SET NULL

Derived total_marks = $\sum$ exam_questions.effective_marks

JSONB payloads

PROCTOR_CONFIG KEY	DEFAULT	MEANING
webcam_enabled	true	
gaze_tracking_enabled	true	
gaze_sensitivity	0.6	0–1, higher is stricter
max_tab_switches	3	Warnings before the session is auto-flagged
flag_on_score	45.0	Suspicion score that flags for review
terminate_on_score	100.0	Suspicion score that ends the session. Validated to exceed flag_on_score
snapshot_interval_seconds	60	10–600
require_fullscreen	true	
block_copy_paste	true	
weights	object	Per-event-type contribution to the suspicion score

Default weights

face_missing 6.0

multiple_faces 15.0

gaze_away 3.0

tab_switch 10.0

window_blur 4.0

fullscreen_exit 8.0

camera_blocked 20.0

paste_attempt 12.0

copy_attempt 6.0

devtools_open 25.0

GRADING_CONFIG KEY	DEFAULT	MEANING
partial_credit_multi_select	false	Award fractional marks for a partially correct multi-select
auto_publish_results	false	
grader_provider	null	Falls back to the app-level setting
grader_model	null	

exam_questions

NO TIMESTAMPS

The pool an exam draws from. A candidate's paper is a subset of these.

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
exam_id	UUID	NO	FK exams.id CASCADE · INDEX
question_id	UUID	NO	FK questions.id RESTRICT
marks_override	float	yes	Per-exam mark value for this question
order_index	int	NO	0

UNIQUE uq_exam_question (exam_id, question_id) · **Derived** effective_marks = marks_override if set, else question.marks

5 Sessions & answers

One session per candidate per exam, enforced in the database. The paper is frozen at start so a server restart mid-exam cannot reshuffle it under the candidate.

exam_sessions

TIMESTAMPS

backend/app/db/models/exam_session.py

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
exam_id	UUID	NO	FK exams.id CASCADE · INDEX
candidate_id	UUID	NO	FK users.id CASCADE · INDEX
paper_seed	varchar(64)	NO	Derived from the exam salt and the candidate; makes the paper reproducible
question_order	jsonb	NO	[{"question_id":..., "option_order": [...]}] — persisted at start
started_at	timestamptz	NO	
expires_at	timestamptz	NO	Server-authoritative deadline; the client clock is never trusted
submitted_at	timestamptz	yes	
last_heartbeat_at	timestamptz	yes	
status	session_status	NO	in_progress, INDEX
suspicion_score	float	NO	0.0 — running sum of proctor event weights
tab_switch_count	int	NO	0
is_flagged	bool	NO	false, INDEX
termination_reason	text	yes	

UNIQUE uq_exam_session_attempt (exam_id, candidate_id) — this is what enforces a single attempt · **Derived** is_open

answers

TIMESTAMPS

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
session_id	UUID	NO	FK exam_sessions.id CASCADE · INDEX
question_id	UUID	NO	FK questions.id RESTRICT
selected_option_ids	jsonb	yes	Objective answers — chosen option ids, stringified
text_answer	text	yes	Subjective answers; capped at 50 000 chars by the API layer
image_object_key	varchar(512)	yes	Object-store key for a handwritten scan
image_thumb_key	varchar(512)	yes	Server-generated, so the examiner queue can render a grid of scans without pulling a dozen multi-megabyte originals through presigned URLs
ocr_text	text	yes	OCR pre-pass over a scan. Advisory only — the examiner reads the image
ocr_confidence	float	yes	
word_count	int	yes	Recorded at save time so grading and analytics never re-tokenise the text
awarded_marks	float	yes	

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
max_marks	float	NO	1.0 — snapshotted from effective_marks at save
grade_status	grade_status	NO	unanswered, INDEX
examiner_comment	text	yes	Capped at 4000 chars by the API layer
graded_by_id	UUID	yes	FK users.id SET NULL
graded_at	timestampz	yes	
answered_at	timestampz	yes	
UNIQUE uq_answer_per_question (session_id, question_id) — autosave is an upsert against this			
Derived latest_ai_evaluation (relationship ordered created_at DESC) · is_answered = any of options, text or image present			

6 Proctoring

One row per behavioural or vision signal. Deliberately without the timestamp mixin — this is the high-volume table, and it is reviewed through its own log stream.

proctor_events

NO TIMESTAMPS

backend/app/db/models/proctor.py

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
session_id	UUID	NO	FK exam_sessions.id CASCADE
event_type	proctor_event_type	NO	
severity	proctor_severity	NO	info
occurred_at	timestamptz	NO	Client clock, echoed for ordering inside a batch
server_received_at	timestamptz	NO	Authoritative timestamp
duration_ms	int	yes	0–3 600 000 at the API layer
weight	float	NO	0.0 — resolved from the exam's proctor_config.weights at intake
metadata	jsonb	yes	Python attribute event_metadata, column named metadata. e.g. {"face_count":2, "yaw":-34.2, "confidence":0.91}
snapshot_object_key	varchar(512)	yes	Webcam frame in the object store

INDEX ix_proctor_events_timeline (session_id, occurred_at) — serves the reviewer's session timeline

7 Grading & results

Machine evaluation is a history, never a verdict. The final tally is one row per session.

ai_evaluations TIMESTAMPS

backend/app/db/models/grading.py

First-pass score produced by the configured grader. Never authoritative on its own — an examiner reviews and may override it. Kept as a history, one row per grading run, so a re-grade does not destroy the earlier verdict.

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
answer_id	UUID	NO	FK answers.id CASCADE · INDEX
provider	varchar(50)	NO	
model	varchar(100)	yes	
score	float	NO	
max_score	float	NO	
justification	text	NO	Shown to the examiner, and to the candidate on a published result
confidence	float	NO	0.0
raw_response	jsonb	yes	Full provider payload, retained for audit, never exposed over the API
error	text	yes	A failed run is recorded, not discarded

results TIMESTAMPS

COLUMN	TYPE	NULL	DEFAULT & CONSTRAINTS
id	UUID	NO	PK
session_id	UUID	NO	FK exam_sessions.id CASCADE · UNIQUE — strict 1:1
total_marks	float	NO	
obtained_marks	float	NO	
percentage	float	NO	
correct_count	int	NO	0
incorrect_count	int	NO	0
unanswered_count	int	NO	0
pending_review_count	int	NO	0 — non-zero blocks publication
published	bool	NO	false, INDEX — gates candidate visibility
published_at	timestamptz	yes	

8 Enum types

All ten are native PostgreSQL enum types, created by the initial migration. Python-side they are `str`, `enum.Enum`, and `values_callable` pins the stored value to the lowercase member value rather than the member name.

`backend/app/db/models/enums.py`

PG TYPE	PYTHON CLASS	VALUES
<code>user_role</code>	<code>UserRole</code>	<code>admin</code> · <code>examiner</code> · <code>candidate</code>
<code>access_status</code>	<code>AccessStatus</code>	<code>pending</code> · <code>approved</code> · <code>revoked</code>
<code>login_access_status</code>	<code>LoginAccessStatus</code>	<code>pending</code> · <code>approved</code> · <code>rejected</code>
<code>question_type</code>	<code>QuestionType</code>	<code>mcq</code> · <code>multi_select</code> · <code>short_answer</code> · <code>long_answer</code> · <code>image_upload</code>
<code>difficulty</code>	<code>Difficulty</code>	<code>easy</code> · <code>medium</code> · <code>hard</code>
<code>exam_status</code>	<code>ExamStatus</code>	<code>draft</code> · <code>published</code> · <code>closed</code>
<code>session_status</code>	<code>SessionStatus</code>	<code>in_progress</code> · <code>submitted</code> · <code>auto_submitted</code> · <code>terminated</code>
<code>grade_status</code>	<code>GradeStatus</code>	<code>unanswered</code> · <code>auto_scored</code> · <code>pending_ai</code> · <code>ai_scored</code> · <code>examiner_reviewed</code>
<code>proctor_event_type</code>	<code>ProctorEventType</code>	<code>face_missing</code> · <code>multiple_faces</code> · <code>gaze_away</code> · <code>tab_switch</code> · <code>window_blur</code> · <code>fullscreen_exit</code> · <code>camera_blocked</code> · <code>paste_attempt</code> · <code>copy_attempt</code> · <code>devtools_open</code>
<code>proctor_severity</code>	<code>ProctorSeverity</code>	<code>info</code> · <code>warning</code> · <code>critical</code>

Question-type sets

Module-level sets that drive the grading fork — objective types are machine-scored at submit; subjective types enter the AI queue and then the examiner queue.

SET	MEMBERS
<code>OBJECTIVE_TYPES</code>	<code>mcq</code> · <code>multi_select</code>
<code>SUBJECTIVE_TYPES</code>	<code>short_answer</code> · <code>long_answer</code> · <code>image_upload</code>

9 Referential integrity map

Two deletion policies, chosen per edge. **CASCADE** where the child has no meaning without its parent. **RESTRICT** where deleting the parent would destroy the record of what was actually asked or answered.

```
users
|--CASCADE--> exam_sessions      (candidate_id)
|--CASCADE--> login_access_requests (candidate_id, UNIQUE - 1:1)
|--CASCADE--> exam_enrollments    (candidate_id)
|--SET NULL-> users.access_changed_by_id (self-referential)
|--SET NULL-> exams.created_by_id
|--SET NULL-> questions.created_by_id
|--SET NULL-> answers.graded_by_id
|--SET NULL-> login_access_requests.reviewed_by_id
`--SET NULL-> exam_enrollments.assigned_by_id

subjects
|--RESTRICT-> questions          (subject_id)
`--RESTRICT-> exams              (subject_id)

questions
|--CASCADE--> question_options    (question_id)
|--RESTRICT-> exam_questions       (question_id)
`--RESTRICT-> answers             (question_id)

exams
|--CASCADE--> exam_questions      (exam_id)
|--CASCADE--> exam_sessions       (exam_id)
`--CASCADE--> exam_enrollments    (exam_id)

exam_sessions
|--CASCADE--> answers            (session_id)
|--CASCADE--> proctor_events      (session_id)
`--CASCADE--> results            (session_id, UNIQUE - 1:1)

answers
`--CASCADE--> ai_evaluations      (answer_id)
```

Uniqueness constraints

CONSTRAINT	TABLE	ENFORCES
uq_exam_session_attempt	exam_sessions	One attempt per candidate per exam
uq_answer_per_question	answers	One answer row per question per session — autosave upserts against it
uq_exam_question	exam_questions	A question appears once in an exam's pool
uq_exam_enrollment	exam_enrollments	One enrolment per candidate per exam
candidate_id UNIQUE	login_access_requests	One standing login request per candidate
session_id UNIQUE	results	One result per session
email UNIQUE	users	Login identity
code UNIQUE	subjects	Subject code

Indexes beyond the primary keys

INDEX	COLUMNS
ix_questions_pool_lookup	(subject_id, question_type, difficulty) — exam pool selection
ix_proctor_events_timeline	(session_id, occurred_at) — reviewer timeline
single-column	users.email, users.role, users.access_status, subjects.code, questions.is_active, exams.status, exam_questions.exam_id, exam_sessions.exam_id, exam_sessions.candidate_id, exam_sessions.status, exam_sessions.is_flagged, answers.session_id, answers.grade_status, ai_evaluations.answer_id, results.published, login_access_requests.candidate_id, login_access_requests.status, exam_enrollments.exam_id, exam_enrollments.candidate_id

10 API schemas — auth & admin

Pydantic v2 models under `backend/app/schemas/`. These are the wire contract, kept separate from persistence: they validate what comes in, and they decide what leaves the server. `ORMModel` sets `from_attributes=True`; the rest are hand-assembled.

`common.py`

MODEL	FIELDS
<code>ORMModel</code>	Base with <code>from_attributes=True</code>
<code>Page[T]</code>	<code>items: list[T]</code> , <code>total</code> , <code>page</code> , <code>page_size</code> ; derived <code>pages</code>
<code>Message</code>	<code>detail: str</code>

`auth.py`

MODEL	FIELDS & VALIDATION
<code>RegisterRequest</code>	<code>email: EmailStr</code> , <code>password</code> (8–128, must contain a letter and a digit), <code>first_name</code> (1–80), <code>last_name</code> (≤80), <code>role = candidate</code> . A validator rejects self-registered <code>admin</code>
<code>UpdateProfileRequest</code>	<code>first_name</code> , <code>last_name</code> — the only self-editable fields. Email is the login identity; role and access are an administrator's decision
<code>LoginRequest</code>	<code>email</code> , <code>password</code>
<code>RefreshRequest</code>	<code>refresh_token</code>
<code>ForgotPasswordRequest</code>	<code>email</code>
<code>ResetPasswordRequest</code>	<code>token</code> , <code>new_password</code> (same strength rule)
<code>UserOut</code>	<code>id</code> , <code>email</code> , <code>first_name</code> , <code>last_name</code> , <code>full_name</code> , <code>role</code> , <code>access_status</code> , <code>is_active</code> , <code>created_at</code> , <code>last_login_at?</code> , <code>access_note?</code> — never <code>password_hash</code>
<code>TokenPair</code>	<code>access_token</code> , <code>refresh_token</code> , <code>token_type = bearer</code> , <code>expires_in</code> , <code>user: UserOut</code> , <code>login_access?</code> — candidates only; null for examiners and admins, who are not login-gated
<code>AccessDecision</code>	<code>access_status</code> , <code>note?</code> (≤500)

`admin.py`

MODEL	FIELDS
<code>UserAdminOut</code>	extends <code>UserOut</code> + <code>exam_count</code> , <code>session_count</code> , <code>access_changed_at?</code> , <code>access_changed_by_email?</code>
<code>AdminUserCreate</code>	<code>email</code> , <code>password</code> (8–128), <code>full_name</code> (2–150), <code>role</code> , <code>access_status = approved</code>
<code>AdminStats</code>	<code>total_users</code> , <code>candidates</code> , <code>examiners</code> , <code>admins</code> , <code>pending_approvals</code> , <code>subjects</code> , <code>questions</code> , <code>exams</code> , <code>published_exams</code> , <code>live_sessions</code> , <code>flagged_sessions</code> , <code>pending_grading</code>
<code>ExaminerStats</code>	<code>my_questions</code> , <code>my_exams</code> , <code>published_exams</code> , <code>live_sessions</code> , <code>flagged_sessions</code> , <code>pending_grading</code> , <code>subjects</code>
<code>CandidateStats</code>	<code>available_exams</code> , <code>completed_exams</code> , <code>published_results</code> , <code>average_percentage?</code> , <code>best_percentage?</code>
<code>CandidateAdminRow</code>	The candidate list admins and examiners both see — <code>id</code> , <code>full_name</code> , <code>email</code> , <code>access_status</code> , <code>is_active</code> , <code>created_at</code> , <code>last_login_at?</code> , <code>attempts</code> , <code>completed</code> , <code>flagged_sessions</code> , <code>average_percentage?</code>
<code>RecentActivity</code>	<code>kind</code> , <code>message</code> , <code>at</code> , <code>severity = info</code>

login_access.py

MODEL	FIELDS
LoginAccessOut	A candidate's own view of where their request stands — <code>status</code> , <code>requested_at</code> , <code>reviewed_at?</code> , <code>review_note?</code>
LoginRequestRow	A row in the reviewer's queue — <code>id</code> , <code>candidate_id</code> , <code>first_name</code> , <code>last_name</code> , <code>full_name</code> , <code>email</code> , <code>status</code> , <code>requested_at</code> , <code>reviewed_at?</code> , <code>reviewed_by_name?</code> , <code>review_note?</code> , <code>enrolled_exams: list[str]</code> — which of the reviewer's own exams this candidate is in, the basis of an examiner's authority to decide
ReviewDecision	<code>note?</code> (≤ 500)
EnrollmentRow	<code>candidate_id</code> , <code>full_name</code> , <code>email</code> , <code>assigned_at</code> , <code>assigned_by_name?</code> , <code>login_access?</code> , <code>has_attempted</code>
EnrollmentCreate	<code>candidate_ids: list[UUID]</code> (1–1000)

11 API schemas — questions & exams

question.py

MODEL	FIELDS & VALIDATION
SubjectCreate	code (2–32), name (2–150), description?
SubjectOut	id, code, name, description?, question_count
OptionIn	text (min 1), is_correct = false, order_index = 0
OptionOut	id, text, order_index — no answer key
OptionOutWithAnswer	extends OptionOut + is_correct
QuestionBase	subject_id, question_type, difficulty = medium, body (min 3), model_answer?, rubric?, marks (>0, default 1.0), negative_marks (≥0), tags?, min_words? (0–100 000), max_words? (1–100 000)
QuestionCreate	extends QuestionBase + options: list[OptionIn]. A model validator delegates to services.validators.validate_question — type-appropriate option counts, at least one correct answer, coherent word bounds
QuestionUpdate	All-optional patch: difficulty, body, model_answer, rubric, marks, negative_marks, tags, min_words, max_words, is_active, options. subject_id and question_type are immutable after creation
QuestionOut	id, subject_id, question_type, difficulty, body, marks, negative_marks, min_words?, max_words?, tags?, is_active, created_at, options: list[OptionOut]
QuestionOutFull	Examiner view — adds model_answer? and rubric?, and upgrades options to OptionOutWithAnswer
BulkQuestionCreate	questions: list[QuestionCreate] (1–500)
BulkQuestionResult	created, failed, errors: list[dict] — partial success is reported per row, not rolled back wholesale

exam.py

MODEL	FIELDS & VALIDATION
SelectionRule	question_type, difficulty?, count (1–200)
SelectionRules	rules: list[SelectionRule] (min 1)
ProctorConfig	webcam_enabled, gaze_tracking_enabled, gaze_sensitivity (0–1), max_tab_switches (0–50), flag_on_score (≥0), terminate_on_score (>0), snapshot_interval_seconds (10–600), require_fullscreen, block_copy_paste, weights?. Validator: terminate_on_score must exceed flag_on_score
GradingConfig	partial_credit_multi_select, auto_publish_results, grader_provider?, grader_model?
ExamCreate	subject_id, title (3–200), description?, duration_minutes (1–600), starts_at, ends_at, selection_rules, randomize, shuffle_options, negative_marking, proctor_config, grading_config, question_ids. Validator: the window must end after it starts, and the duration must fit inside it
ExamUpdate	All-optional patch over the same fields; subject_id is immutable
ExamOut	The persisted fields plus pool_size, total_questions, subject_name?. paper_salt is never exposed
ExamPoolCheck	can_publish, problems: list[str], pool_size, required_count — the pre-publish gate

MODEL	FIELDS & VALIDATION
PaperPreviewEntry	question_id, body, question_type, difficulty, marks, option_order
PaperPreview	seed, candidate_id, total_marks, entries — lets an examiner reproduce exactly the paper a given candidate would receive

12 API schemas — sessions, grading, proctoring

exam_session.py

MODEL	FIELDS & VALIDATION
PaperQuestion	What the candidate sees — never carries the answer key . <code>question_id</code> , <code>question_type</code> , <code>difficulty</code> , <code>body</code> , <code>marks</code> , <code>negative_marks</code> , <code>min_words?</code> , <code>max_words?</code> , <code>options: list[OptionOut]</code> , <code>saved_option_ids</code> , <code>saved_text?</code> , <code>saved_image_url?</code> , <code>saved_word_count?</code>
ExamSessionOut	<code>session_id</code> , <code>exam_id</code> , <code>exam_title</code> , <code>status</code> , <code>started_at</code> , <code>expires_at</code> , <code>server_time</code> , <code>seconds_remaining</code> , <code>total_marks</code> , <code>questions</code> , <code>proctor_config</code> , <code>exam_token?</code>
AnswerSave	<code>selected_option_ids?</code> , <code>text_answer?</code> (≤50 000). Validator requires at least one of the two
AnswerSaved	<code>question_id</code> , <code>saved_at</code> , <code>seconds_remaining</code> , <code>word_count?</code> — the server's count, so the runner's counter is corrected from the authority rather than drifting on its own tokenisation
HeartbeatOut	<code>server_time</code> , <code>seconds_remaining</code> , <code>status</code> , <code>suspicion_score</code> , <code>warnings</code> , <code>exam_token?</code>
SubmitOut	<code>session_id</code> , <code>status</code> , <code>submitted_at</code> , <code>auto_scored</code> , <code>pending_review</code> , <code>message</code>
QuestionResult	<code>question_id</code> , <code>body</code> , <code>question_type</code> , <code>marks</code> , <code>awarded_marks</code> , <code>grade_status</code> , <code>is_correct?</code> , <code>your_answer?</code> , <code>correct_answer?</code> , <code>examiner_comment?</code> , <code>ai_justification?</code>
ResultOut	Mirrors the <code>results</code> row
ResultDetail	<code>result</code> , <code>exam_title</code> , <code>subject_name</code> , <code>candidate_name</code> , <code>submitted_at</code> , <code>questions: list[QuestionResult]</code>
CandidateExamCard	A row in the candidate's "my exams" list — <code>exam_id</code> , <code>title</code> , <code>subject_name</code> , <code>duration_minutes</code> , <code>starts_at</code> , <code>ends_at</code> , <code>total_questions</code> , <code>session_status?</code> , <code>session_id?</code> , <code>result_id?</code> , <code>result_published</code> , <code>can_start</code> , <code>reason?</code> — why not, when <code>can_start</code> is false

grading.py

MODEL	FIELDS
AiEvaluationOut	<code>provider</code> , <code>model?</code> , <code>score</code> , <code>max_score</code> , <code>justification</code> , <code>confidence</code> , <code>created_at</code> , <code>error?</code> — <code>raw_response</code> stays in the database
GradingQueueItem	<code>answer_id</code> , <code>session_id</code> , <code>exam_id</code> , <code>exam_title</code> , <code>candidate_name</code> , <code>candidate_email</code> , <code>question_id</code> , <code>question_body</code> , <code>question_type</code> , <code>model_answer?</code> , <code>rubric?</code> , <code>text_answer?</code> , <code>word_count?</code> , <code>words_below_minimum</code> , <code>image_url?</code> , <code>image_thumb_url?</code> , <code>ocr_text?</code> , <code>ocr_confidence?</code> , <code>max_marks</code> , <code>awarded_marks?</code> , <code>grade_status</code> , <code>examiner_comment?</code> , <code>ai_evaluation?</code> , <code>submitted_at?</code>
GradeOverride	<code>awarded_marks</code> (≥0), <code>comment?</code> (≤4000)
RegradeRequest	<code>answer_ids?</code> or <code>session_id?</code> — target a set of answers or a whole session
GradingSummary	<code>exam_id</code> , <code>exam_title</code> , <code>total_sessions</code> , <code>submitted_sessions</code> , <code>pending_review</code> , <code>reviewed</code> , <code>results_published</code>

`words_below_minimum` counts how far an answer falls under the question's minimum, when the examiner set one. The platform never penalises for it — it is surfaced so the examiner can. `ocr_text` is Tesseract's read of a handwritten scan, and is advisory: the examiner reads the image.

proctor.py

MODEL	FIELDS
ProctorEventIn	<code>event_type</code> , <code>occurred_at</code> , <code>severity?</code> , <code>duration_ms?</code> (0–3 600 000), <code>metadata?</code>

MODEL	FIELDS
ProctorBatchIn	events (max 200) — one flush of the client-side buffer, roughly 10 s worth
ProctorBatchOut	accepted, suspicion_score, tab_switch_count, is_flagged, terminated, warnings
SnapshotOut	snapshot_key, event_id?
ProctorEventOut	id, event_type, severity, occurred_at, server_received_at, duration_ms?, weight, metadata?, snapshot_url? (presigned)
ProctorReview	session_id, candidate_name, candidate_email, exam_title, status, started_at, submitted_at, suspicion_score, tab_switch_count, is_flagged, termination_reason?, breakdown: dict[str, float] — score contribution per event type — and events

13 Migrations

Alembic, linear history. `backend/alembic/versions/`

REVISION	SLUG	INTRODUCES
03ad33e6bd09	<code>initial_schema</code>	Nine enum types and the core tables: <code>users</code> , <code>subjects</code> , <code>questions</code> , <code>question_options</code> , <code>exams</code> , <code>exam_questions</code> , <code>exam_sessions</code> , <code>answers</code> , <code>proctor_events</code> , <code>ai_evaluations</code> , <code>results</code>
f0c6eeb2272c	<code>login_access_requests_exam_enrollments_...</code>	The <code>login_access_status</code> enum, plus <code>login_access_requests</code> and <code>exam_enrollments</code> — the login gate and per-exam assignment
a17b4c9de204	<code>word_bounds_thumbnails_ocr</code>	<code>questions.min_words / max_words</code> ; <code>answers.word_count</code> , <code>image_thumb_key</code> , <code>ocr_text</code> , <code>ocr_confidence</code>

Working commands

COMMAND	EFFECT
<code>python -m alembic upgrade head</code>	Apply all pending migrations
<code>python -m alembic revision --autogenerate -m "..."</code>	Diff the models against the database and draft a revision
<code>python -m app.seed</code>	Load development fixtures

Run from `backend/` with the `venv` interpreter, `.venv/Scripts/python`. Truncation order for a data reset — children first: `ai_evaluations`, `results`, `proctor_events`, `answers`, `exam_sessions`, `exam_questions`, `exams`, `question_options`, `questions`, `subjects`, `users`.